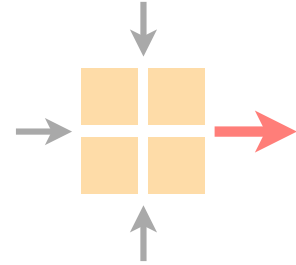


Communication Networks

Spring 2024



Laurent Vanbever

nsg.ee.ethz.ch

29 April 2024

D-ITET

Materials inspired from Scott Shenker & Jennifer Rexford

What problems should be solved by the transport layer?

Deliver *correct* data to the *correct* application process

- IP packets can arrive corrupted
- Nothing in the IP header identifies the source/destination application
- *Transport needs to detect corruption and multiplex incoming data*

Provide a "file-like" *bytestream* abstraction for applications

- Applications write/read bytes to/from the network, as if it was a file
- Network transmits IP packets
- *Transport needs to divide the byte stream into packets on the sender; and reconstruct the byte stream on the receiver.*

Ensure reliability

Prevent overloading the receiver

Prevent overloading the network

How do we solve these problems?

Corruption: Checksum

Demultiplexing: Identifier for application process

- Going from host-to-host (IP) to process-to-process
- Adding application identifiers ("ports") to the transport header

Translating between bytestreams and packets:

- Do segmentation and reassembly

Reliability: ACKs and all that stuff

Not overloading receiver: “Flow Control”

- Limit data in receiver’s buffer

Not overloading network: “Congestion Control”

Important Context: Sockets and Ports

Sockets: an operating system abstraction which provides applications with a uniform (protocol-independent) interface to the network

Ports: a networking abstraction

- This is not a port on a switch (which is an interface)
- Think of it as a *logical interface* on a host

Sockets

A socket is a software abstraction by which an application process exchanges data with the (transport layer in the) operating system

- `socketID = socket(..., socket.TYPE)`
- `socketID.sendto(message, ...)`
- `socketID.recvfrom(...)`

Two important types of sockets

- UDP socket: TYPE is `SOCK_DGRAM`
- TCP socket: TYPE is `SOCK_STREAM`

Ports

Problem: which app (socket) gets which packets

Solution: Use ports as transport layer identifier

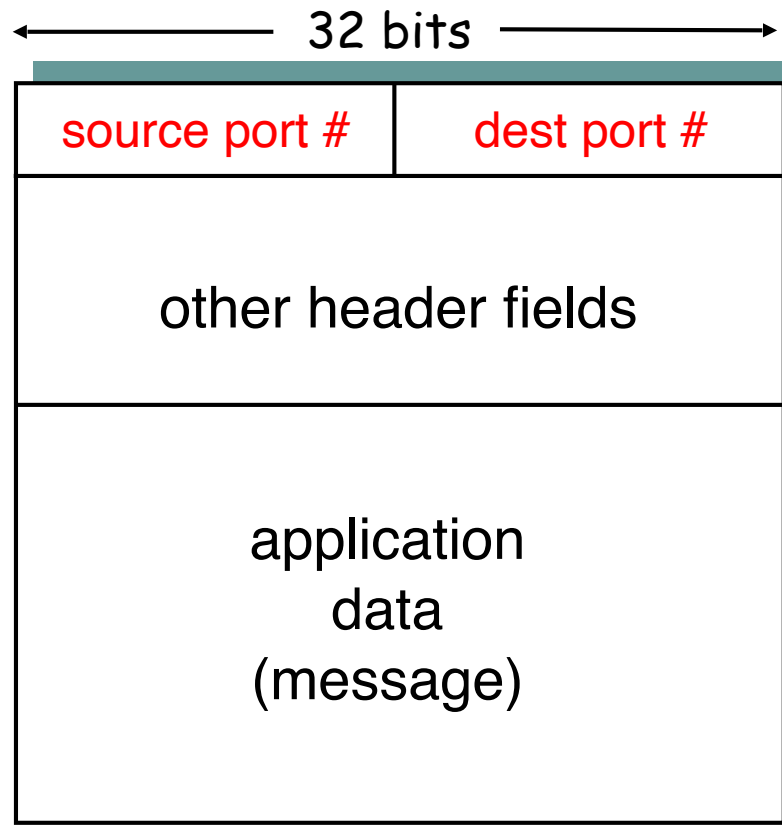
- Ports are 16-bits integers
- Transport headers contain source/destination port numbers
- OS stores the mapping between sockets and ports

Multiplexing and Demultiplexing

Host receives IP packets containing segments

- Each packet has source and destination IP address,
- Each segment has source and destination port number

OS uses IP addresses and port numbers to direct the segment to appropriate socket



The OS identifies TCP/UDP sockets using a 5-tuple:
(src IP, src port, dst IP, dest port, protocol)

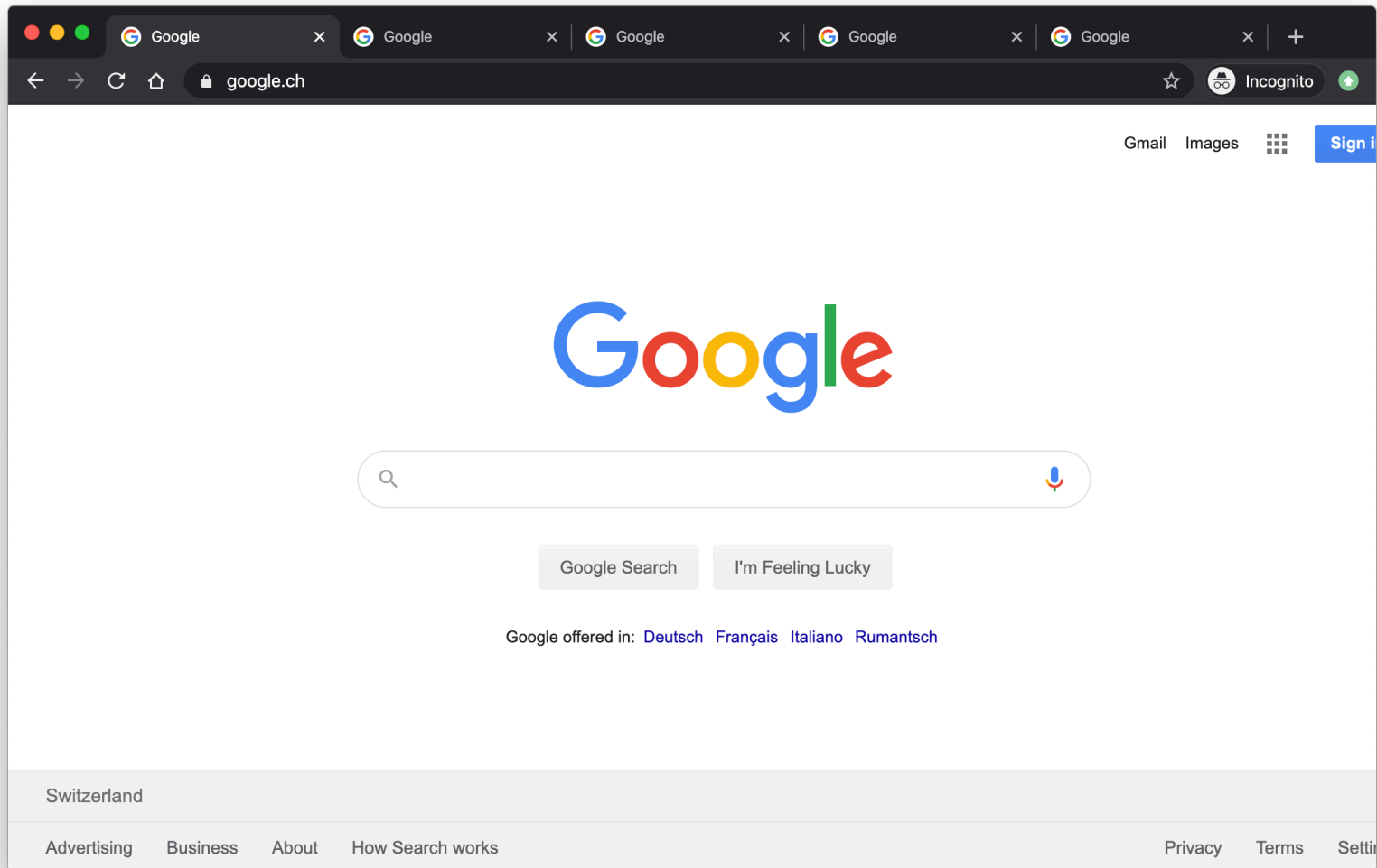
1

2

3

4

5



Let's say you open 5 tabs to google.ch

Your IP: 129.132.19.1

Google's IP: 172.217.168.3

Client OS		src IP	src port	dest IP	dest port	proto	
socket 1 2 3 4 5 	1	129.132.19.1	54001	172.217.168.3	443	TCP	
	2	129.132.19.1	55240	172.217.168.3	443	TCP	
	3	129.132.19.1	48472	172.217.168.3	443	TCP	
	4	129.132.19.1	35456	172.217.168.3	443	TCP	
	5	129.132.19.1	42001	172.217.168.3	443	TCP	
Server OS		src IP	src port	dest IP	dest port	proto	
socket 1 2 3 4 5 	1	172.217.168.3	443	129.132.19.1	54001	TCP	
	2	172.217.168.3	443	129.132.19.1	55240	TCP	
	3	172.217.168.3	443	129.132.19.1	48472	TCP	
	4	172.217.168.3	443	129.132.19.1	35456	TCP	
	5	172.217.168.3	443	129.132.19.1	42001	TCP	

More on Ports

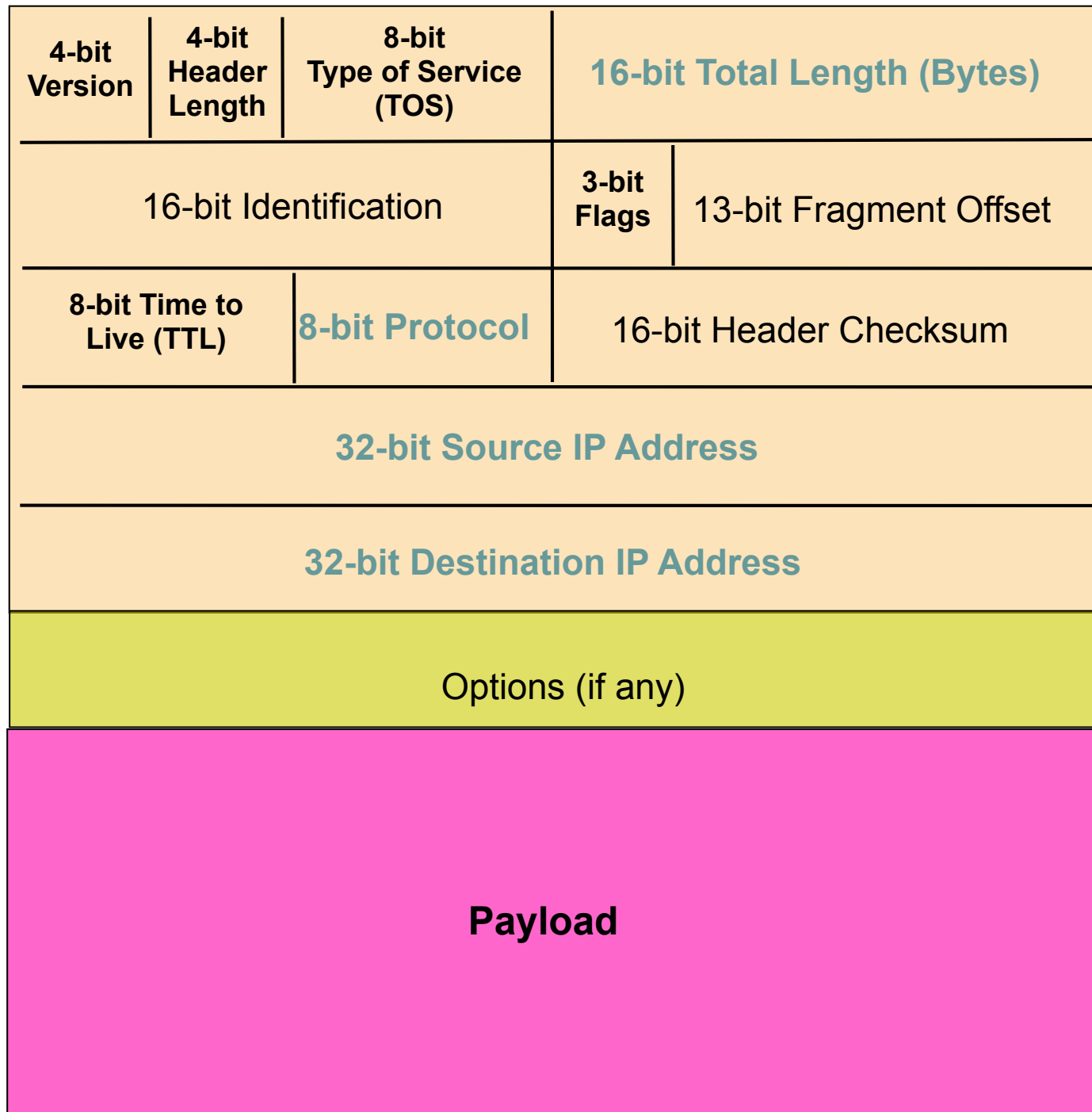
Separate 16-bit port address space for UDP, TCP

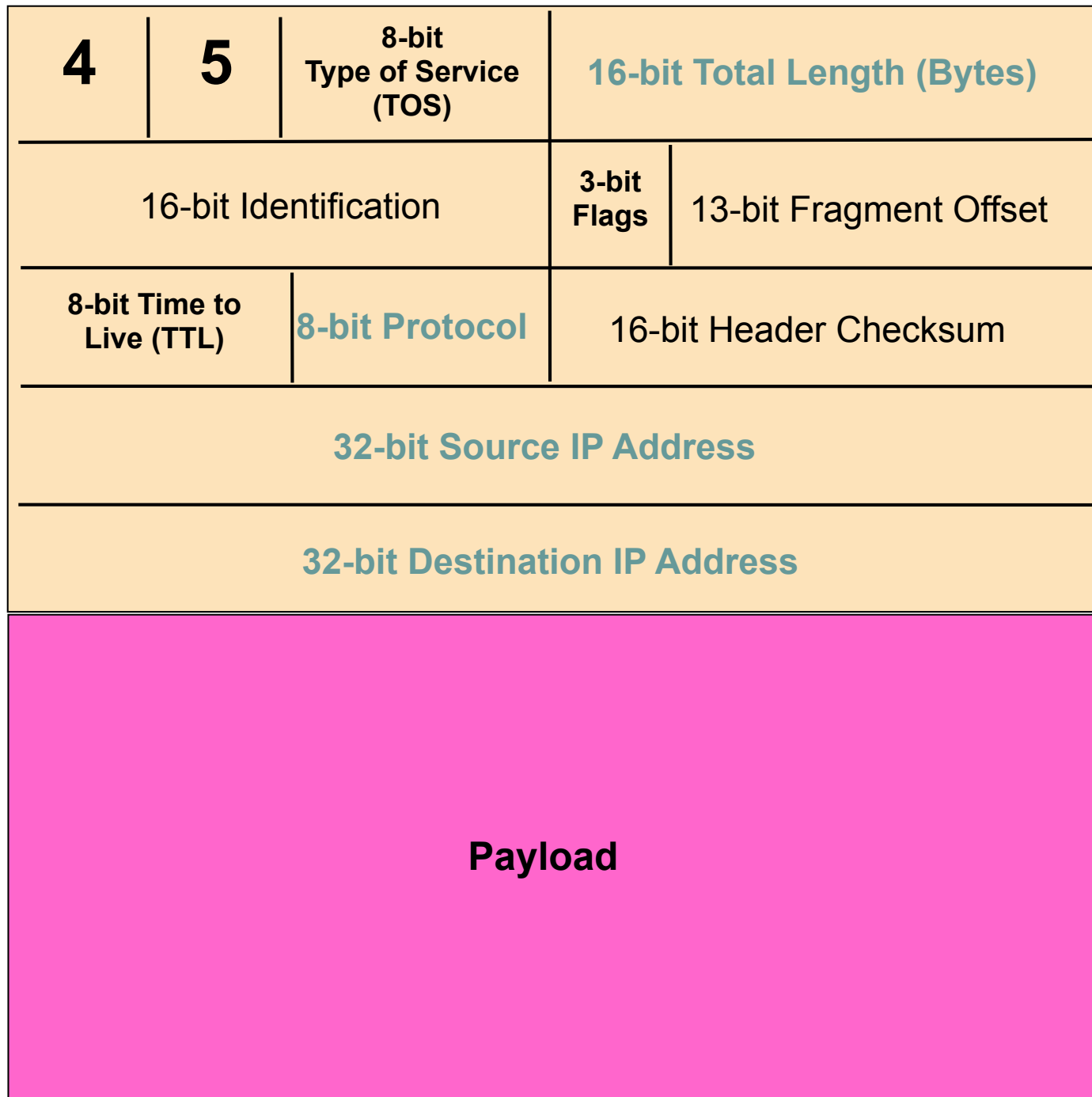
“Well-known” ports (0-1023)

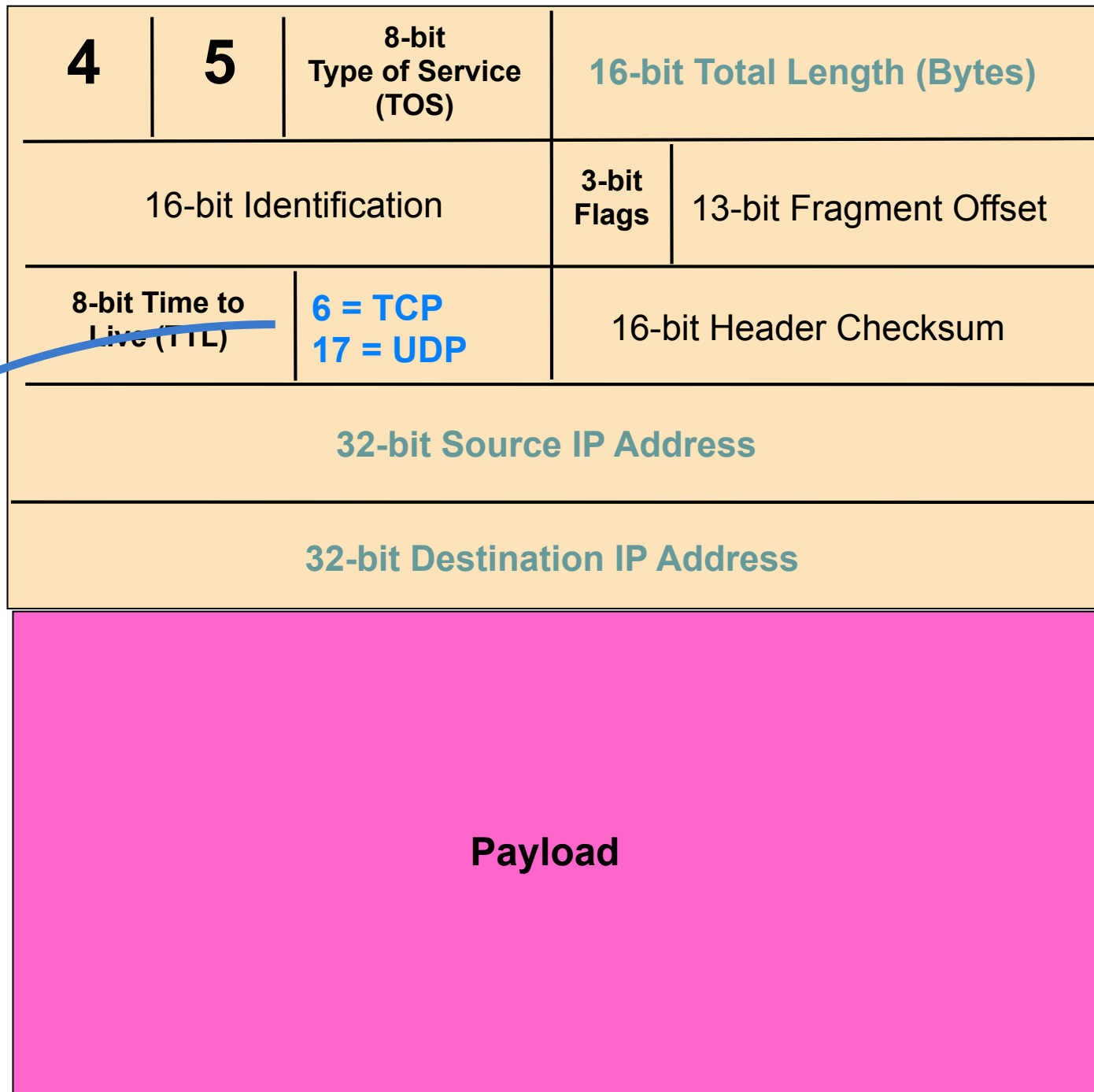
- Agreement on which services run on these ports
- *e.g.*, ssh:22, http:80, https:443
- Client (app) knows appropriate port on server
- Services can listen on well-known port

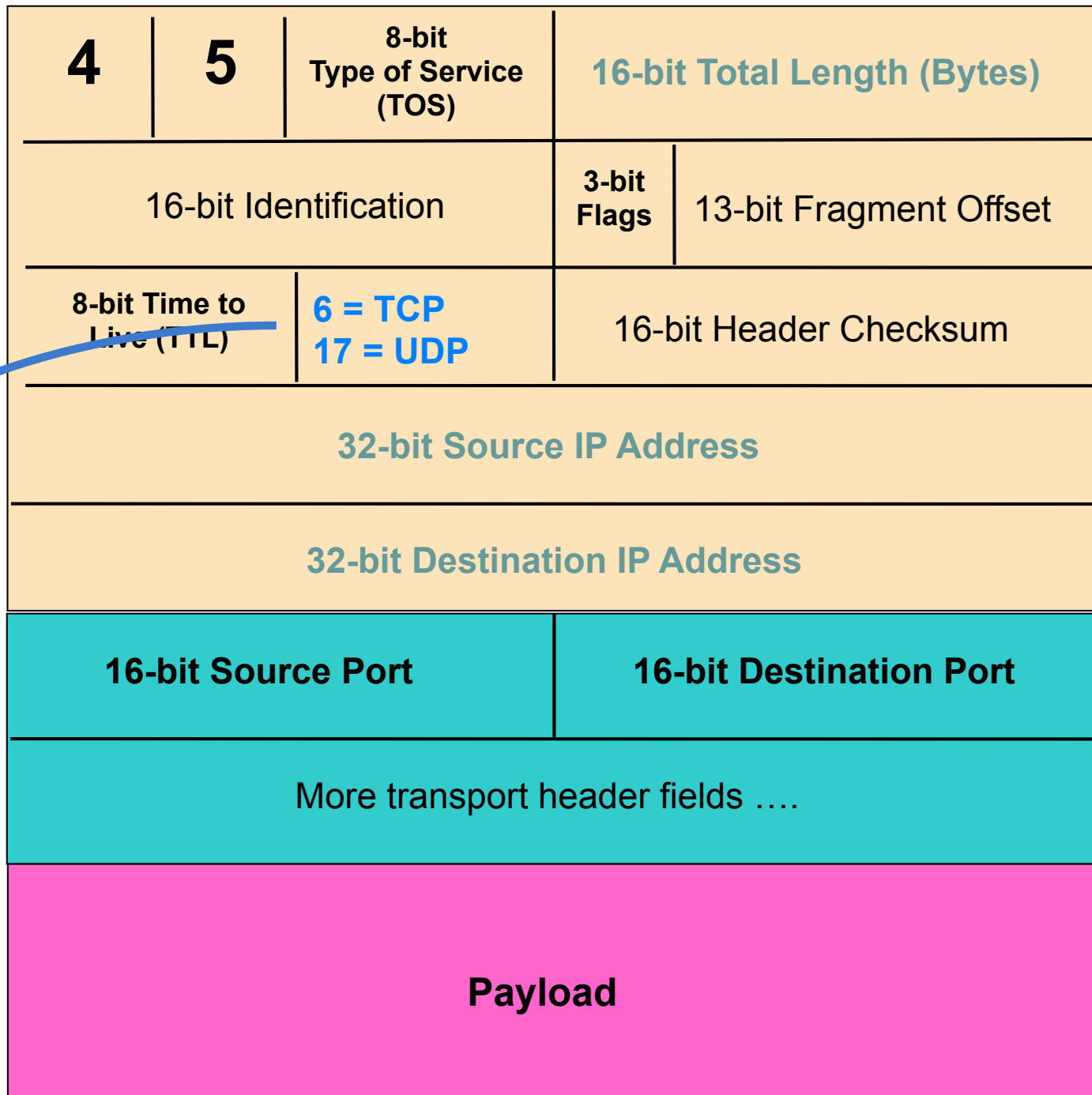
Ephemeral ports (most 1024-65535):

- Given to clients (at random)









UDP

UDP: Datagram messaging service

- UDP provides a **connectionless, unreliable** transport service
- UDP provides **only two services** to the application layer
 - Multiplexing/Demultiplexing amongst processes
 - Discarding corrupted packets (optionally)
- UDP is a simple extension of best-effort IP
 - UDP specification (RFC 768, 1980) is only 2-pages long

SRC port	DST port
checksum	length
DATA	

Why would anyone use UDP?

Fine-grained control over what data is sent and when

- UDP sends as soon as the application writes

No connection set-up delay

- UDP just blasts away without any formal preliminaries
- ... which avoids introducing any unnecessary delays

No connection state

- No buffers, parameters, sequence #s, timers ...
- ... making it easier to handle many active clients at once

Small packet header overhead

- UDP header is only 8-bytes long

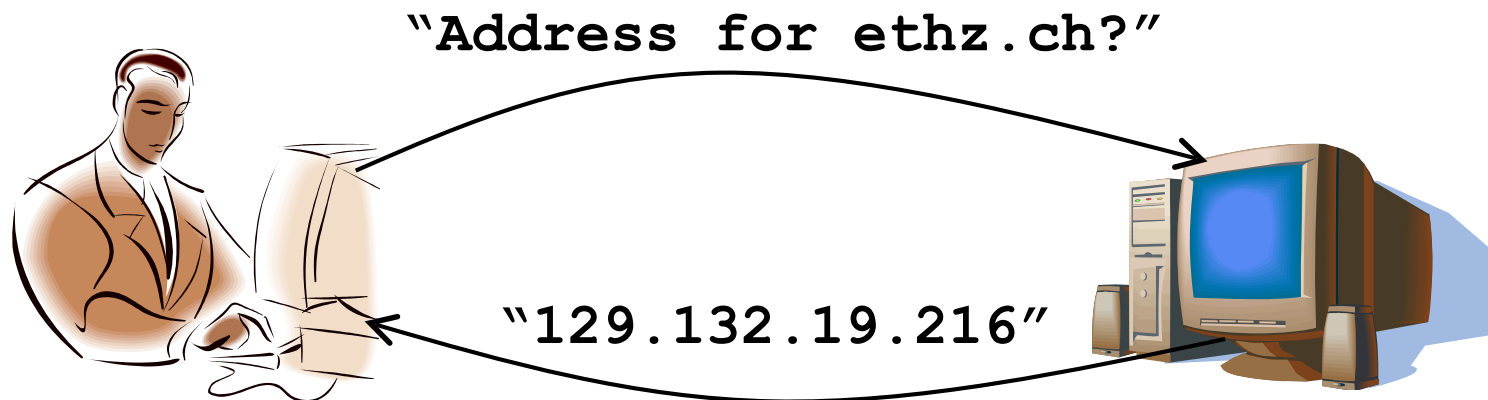
Popular applications that use UDP

Multimedia streaming

- Retransmitting lost/corrupted packets often pointless: by the time the packet is retransmitted, it's too late
 - telephone calls, video conferencing, gaming, IPTV, ...

Simple query-response protocols

- Establishing a connection would be an overkill
 - Domain Name Systems (DNS), DHCP, etc.



TCP

TCP: Reliable, in-order delivery

TCP provides a **connection-oriented, reliable, bytestream** transport service

- Ensures byte stream (eventually) arrives intact
- Explicit set-up and tear-down of TCP session
- Sends and receives a stream of bytes, not messages

What UDP provides, plus:

- Retransmission of lost and corrupted packets
- Flow control (to not overflow receiver)
- Congestion control (to not overload network)
- “Connection” set-up & tear-down

Basic Components of Reliability

ACKs

- Can't be reliable without knowing whether data has arrived
- ***TCP uses byte sequence numbers to identify payloads***

Checksums

- Can't be reliable without knowing whether data is corrupted
- ***TCP does checksum over TCP and (parts of) IP headers***

Timeouts and retransmissions

- Can't be reliable without retransmitting lost/corrupted data
- ***TCP retransmits based on timeouts and duplicate ACKs***
- *Timeout based on estimate of RTT*

Other TCP Design Decisions

Sliding window flow control

- Allow W contiguous bytes to be in flight

Cumulative acknowledgements

- Selective ACKs (full information) also supported

Single timer set after each payload is ACKed

- Timer is effectively for the “next expected payload”
- When timer goes off, resend that payload and wait
 - And double timeout period

Various tricks related to “fast retransmit”

- Using duplicate ACKs to trigger retransmission

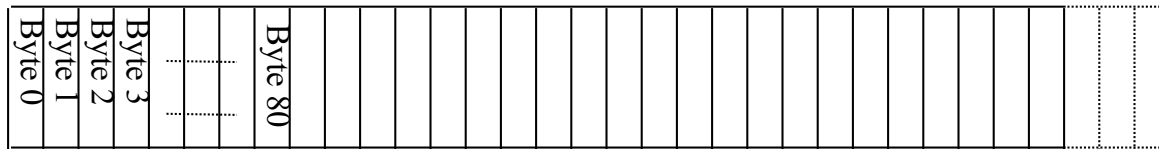
TCP Header

Source port		Destination port	
Sequence number			
Acknowledgment			
HdrLen	0	Flags	Advertised window
Checksum		Urgent pointer	
Options (variable)			
Data			

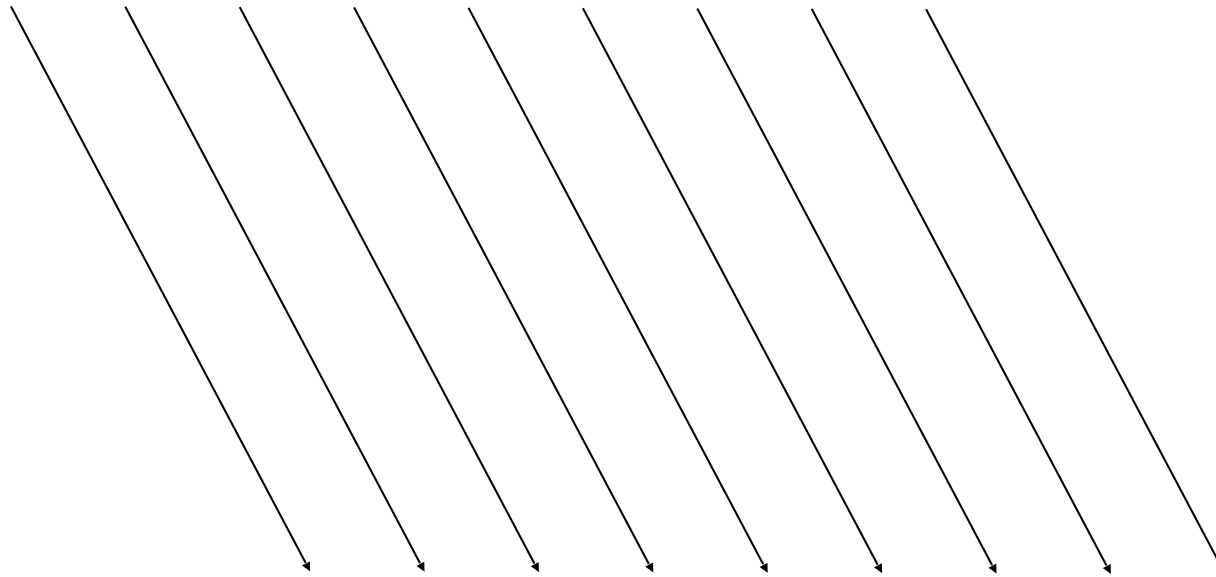
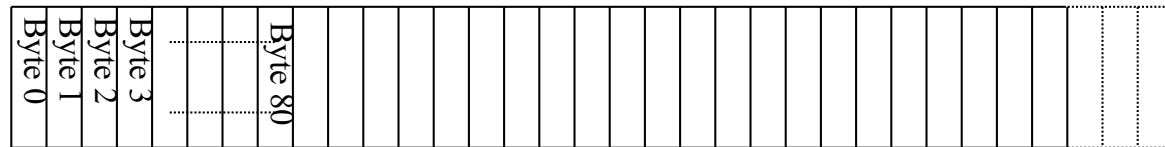
Segments and Sequence Numbers

TCP “Stream of Bytes” service...

Host A

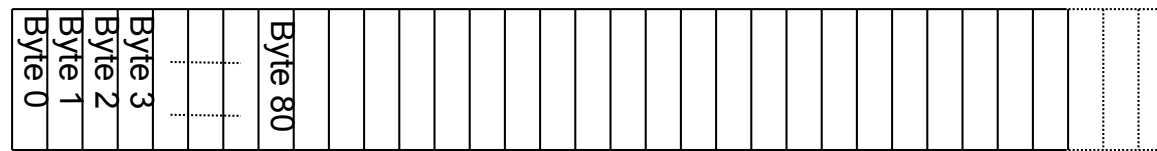


Host B



... emulated using TCP “Segments”

Host A



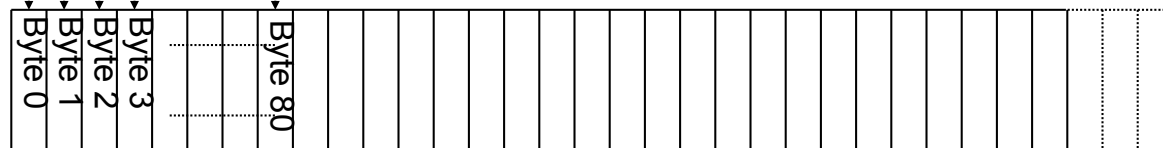
TCP Data

Segment sent when:

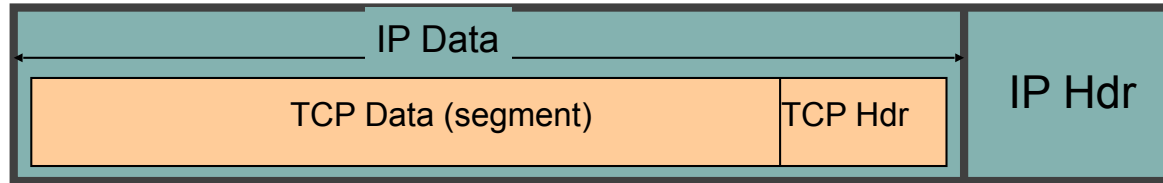
1. Segment full (Max Segment Size)
2. Not full, but times out
3. "Pushed" by the application

TCP Data

Host B



TCP Segment



IP packet

- No bigger than Maximum Transmission Unit ([MTU](#))
- E.g., 1500 bytes with Ethernet (IP header $\geq$ 20 bytes long)

TCP packet

- IP packet with a TCP header and data inside
- TCP header $\geq$ 20 bytes long

TCP segment

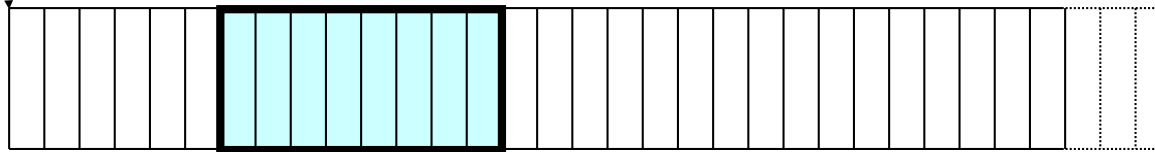
- No more than [Maximum Segment Size](#) (MSS) bytes
- E.g., up to 1460 consecutive bytes from the stream
- $MSS = MTU - (IP \text{ header}) - (TCP \text{ header})$

Sequence Numbers

ISN (initial sequence number)

k bytes

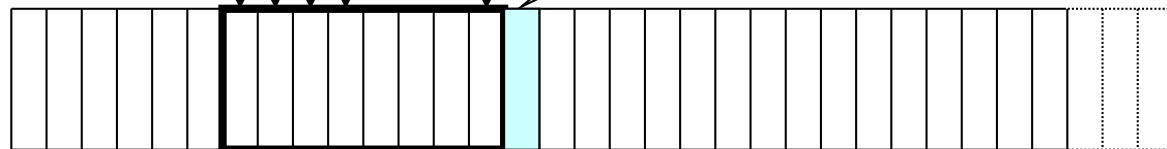
Host A



Sequence number
= 1st byte in segment =
 $ISN + k$



Host B



ACK sequence number
= next expected byte
= $seqno + \text{length}(\text{data})$

ACKing and Sequence Numbers

Sender sends packet

- Data starts with sequence number X
- Packet contains B bytes
 - $X, X+1, X+2, \dots, X+B-1$

Upon receipt of packet, receiver sends an ACK

- If all data prior to X already received:
 - ACK acknowledges $X+B$ (because that is next expected byte)
- If highest contiguous byte received is smaller value Y
 - ACK acknowledges $Y+1$
 - Even if this has been ACKed before

Normal Pattern

Sender: seqno= X , length= B

Receiver: ACK= $X+B$

Sender: seqno= $X+B$, length= B

Receiver: ACK= $X+2B$

Sender: seqno= $X+2B$, length= B

...

Seqno of next packet is same as last ACK field

TCP Header

Source port		Destination port	
Sequence number			
Acknowledgment			
HdrLen	0	Flags	Advertised window
Checksum		Urgent pointer	
Options (variable)			
Data			

Sliding Window Flow Control

Advertised Window: W

- Can send W bytes beyond the next expected byte

Receiver uses W to prevent sender from overflowing buffer

Limits number of bytes sender can have in flight

Advertised Window Limits Rate

Sender can send no faster than W/RTT bytes/sec

Receiver only advertises more space when it has consumed old arriving data

In original TCP design, that was the **sole** protocol mechanism controlling sender's rate

What's missing? Not overloading the network, aka congestion control

Implementing Sliding Window

Both sender & receiver maintain a **window**

- Sender: not yet ACK'ed
- Receiver: not yet delivered to application

Left edge of window:

- Sender: beginning of **unacknowledged** data
- Receiver: beginning of **undelivered** data

For the sender:

- Window size = maximum amount of data in flight

For the receiver:

- Window size = maximum amount of undelivered data

Sliding Window Summary

Sender: window **advances** when new data ack'd

Receiver: window advances as receiving process **consumes** data

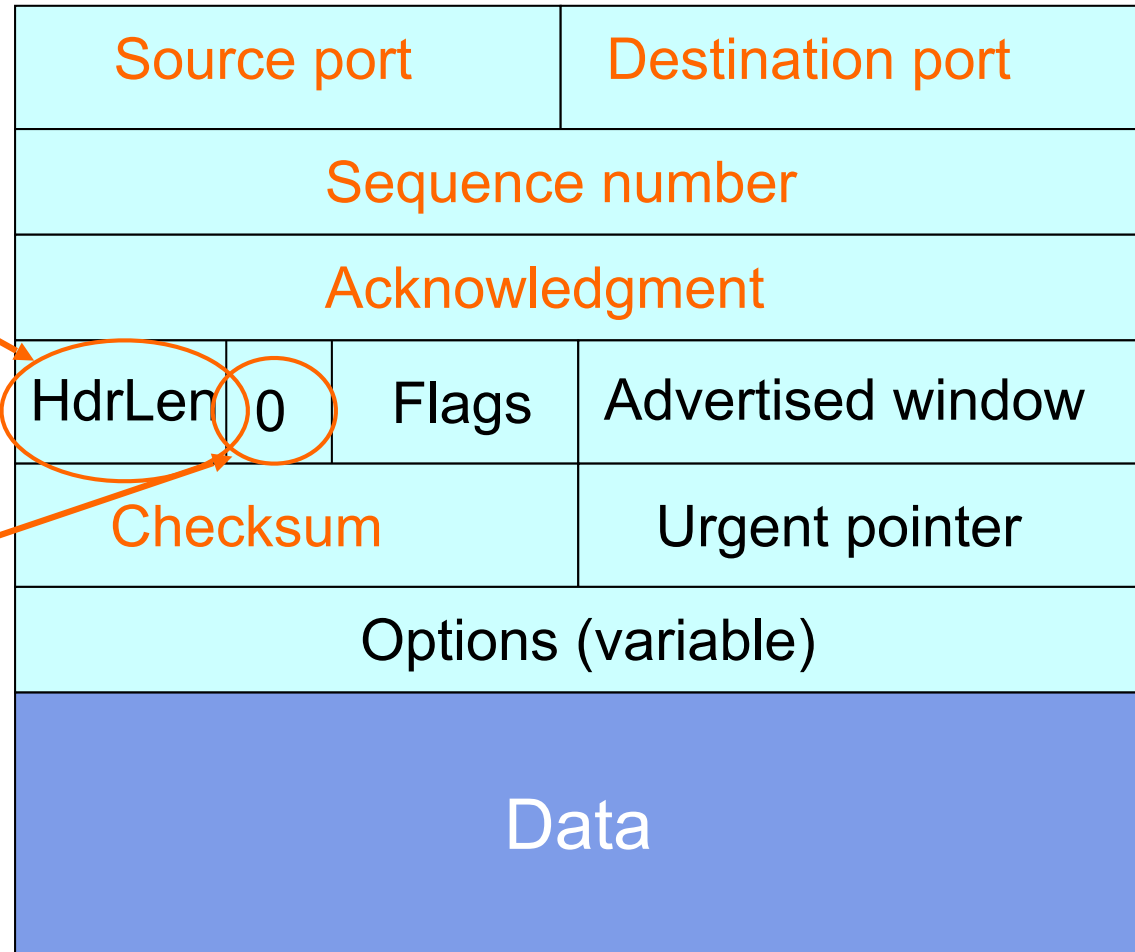
Receiver **advertises** to the sender where the receiver window currently ends (“righthand edge”)

- Sender agrees not to exceed this amount
- It makes sure by setting its own window size to a value that can't send beyond the receiver's righthand edge

TCP Header: What's left?

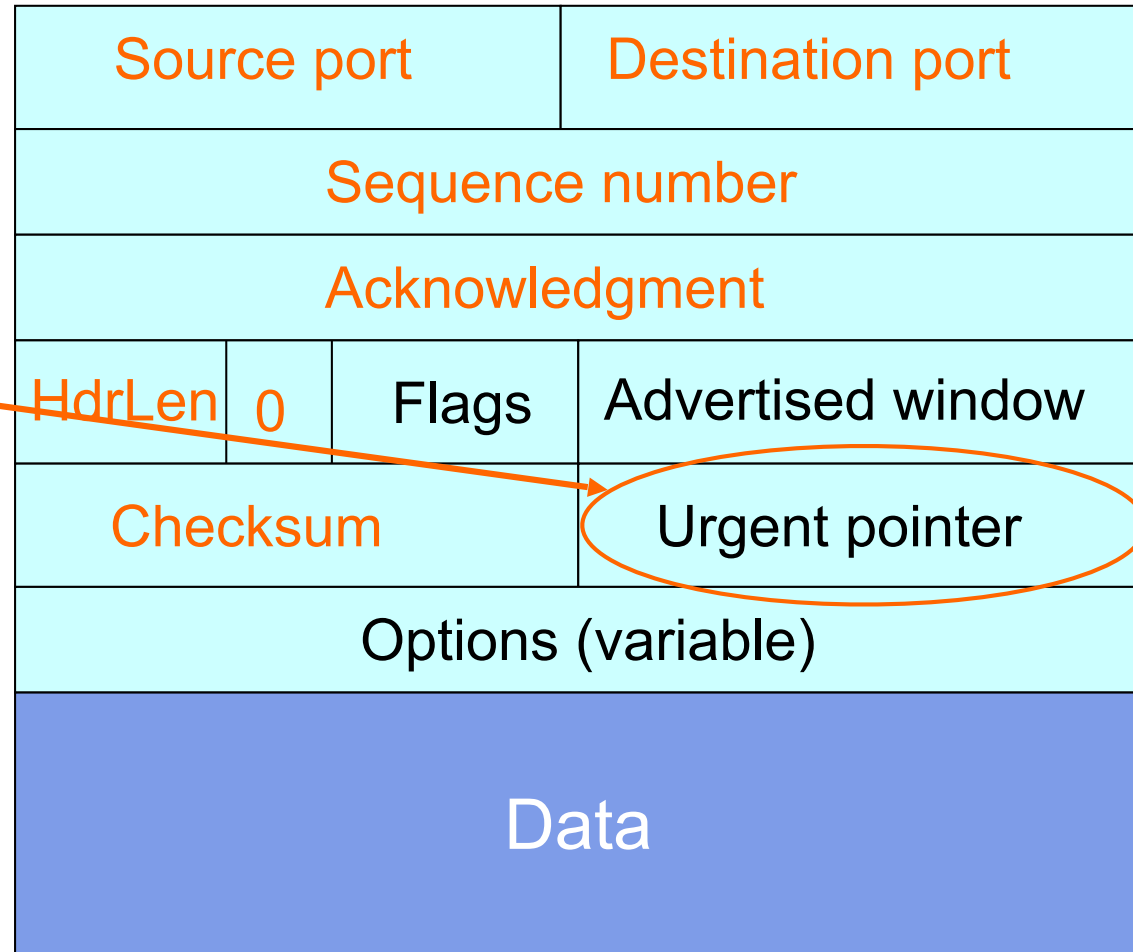
Number of 4-byte words in TCP header;
5 = no options

“Must Be Zero”
6 bits reserved



TCP Header: What's left?

Used with **URG** flag to indicate urgent data (not discussed further)



TCP Header: What's left?

Source port		Destination port	
Sequence number			
Acknowledgment			
HdrLen	0	Flags	Advertised window
Checksum		Urgent pointer	
Options (variable)			
Data			

TCP Connection Establishment and Initial Sequence Numbers

Initial Sequence Number (ISN)

Sequence number for the very first byte

- E.g., Why not just use ISN = 0?

Practical issue

- IP addresses and port #s uniquely identify a connection
- Eventually, though, these port #s do get **used again**
- ... small chance an old packet is **still in flight**

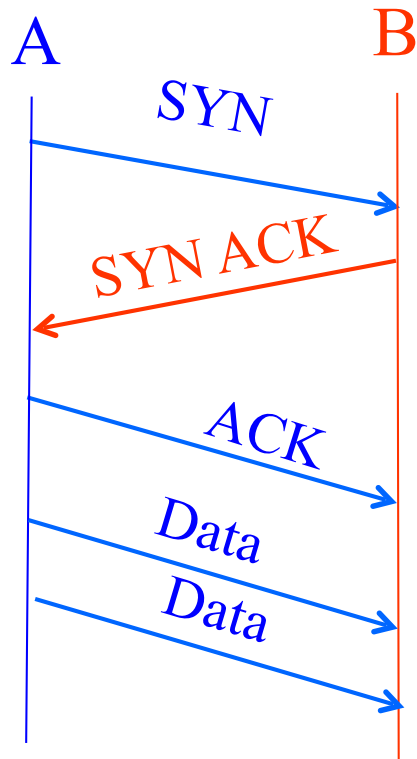
TCP therefore **requires** changing ISN

- initially set from 32-bit clock that ticks every 4 microseconds
- now drawn from a pseudo random number generator (security)

To establish a connection, hosts exchange ISNs

- **How does this help?**

Establishing a TCP Connection



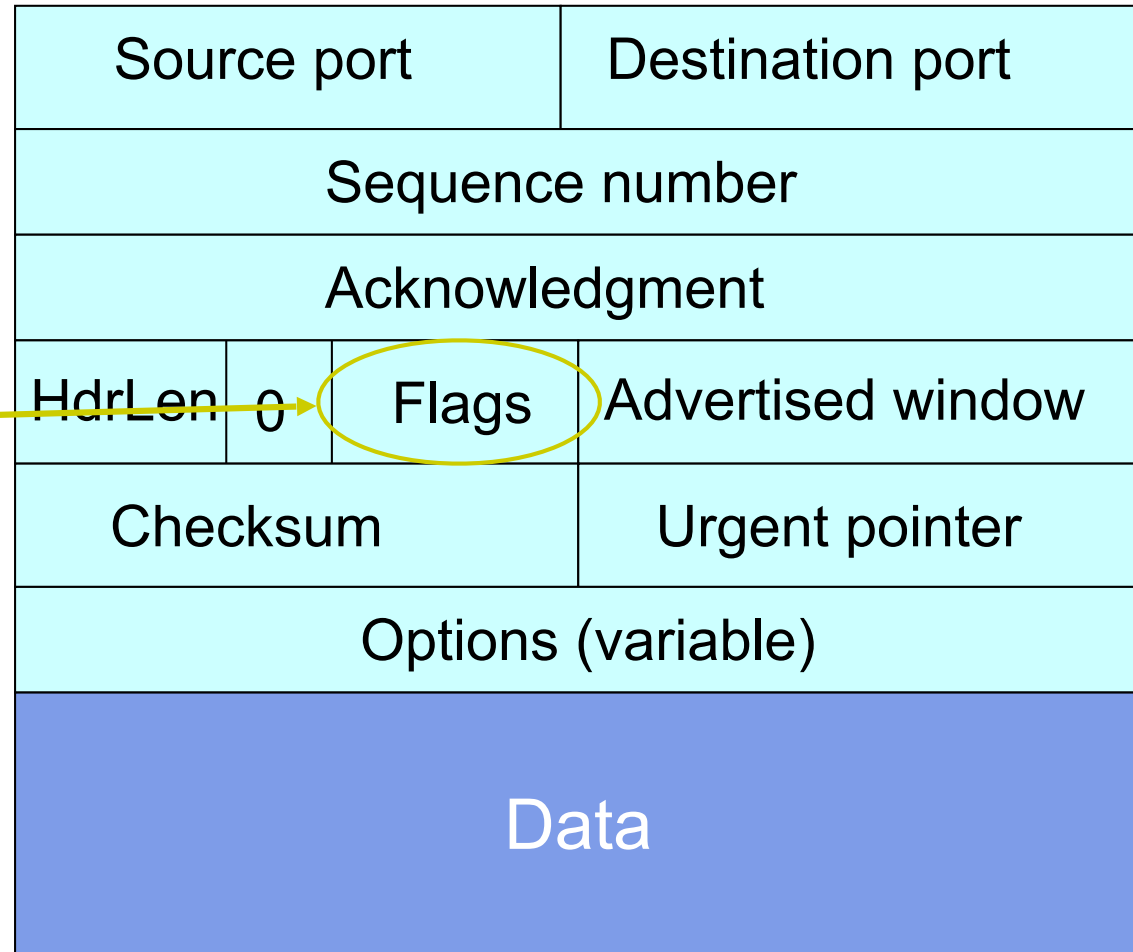
Each host tells
its ISN to the
other host.

Three-way handshake to establish connection

- Host A sends a **SYN** (open; “synchronize sequence numbers”)
- Host B returns a SYN acknowledgment (**SYN ACK**)
- Host A sends an **ACK** to acknowledge the SYN ACK

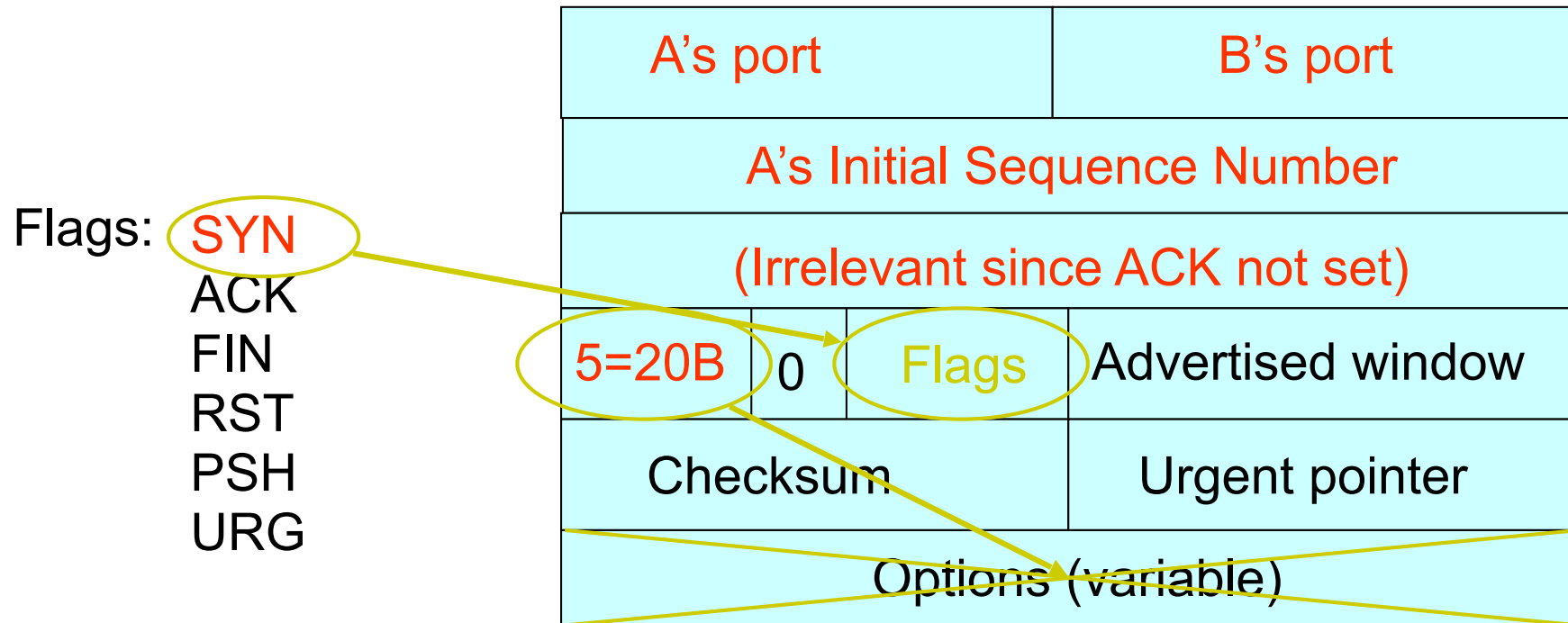
TCP Header

Flags: SYN
ACK
FIN
RST
PSH
URG



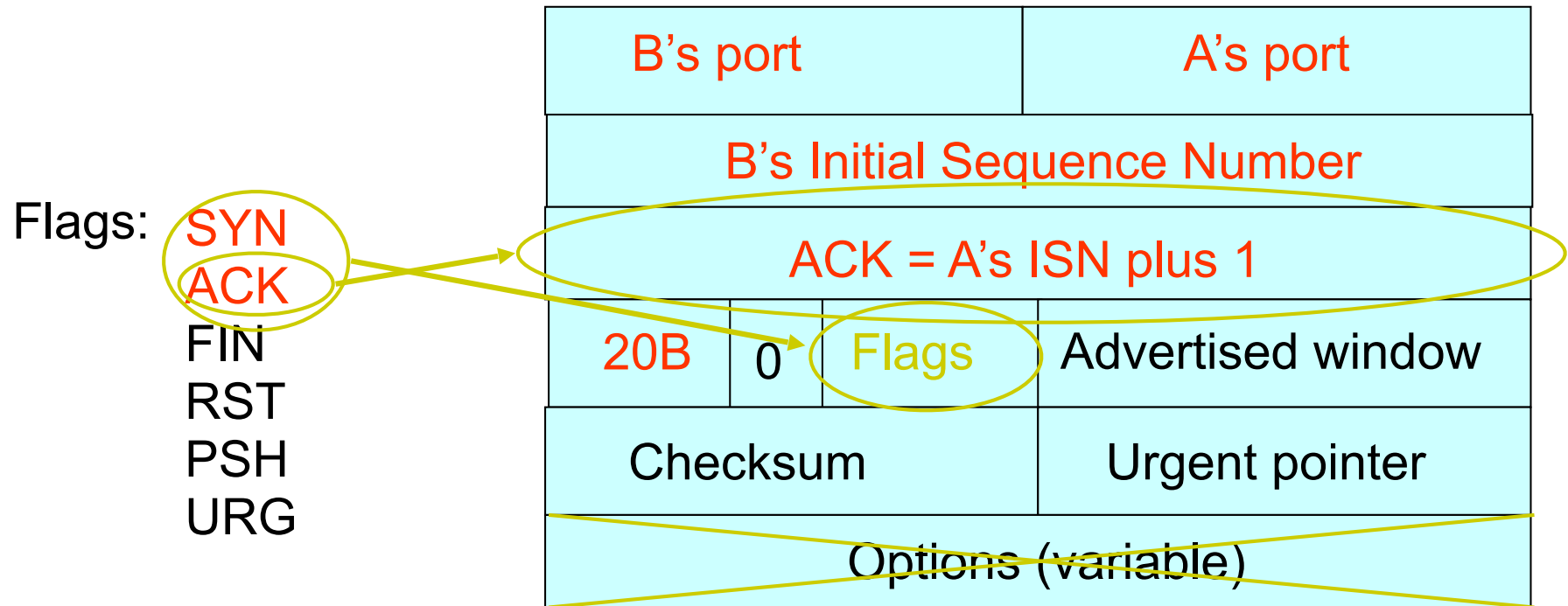
See /usr/include/netinet/tcp.h on Unix Systems

Step 1: A's Initial SYN Packet



A tells B it wants to open a connection...

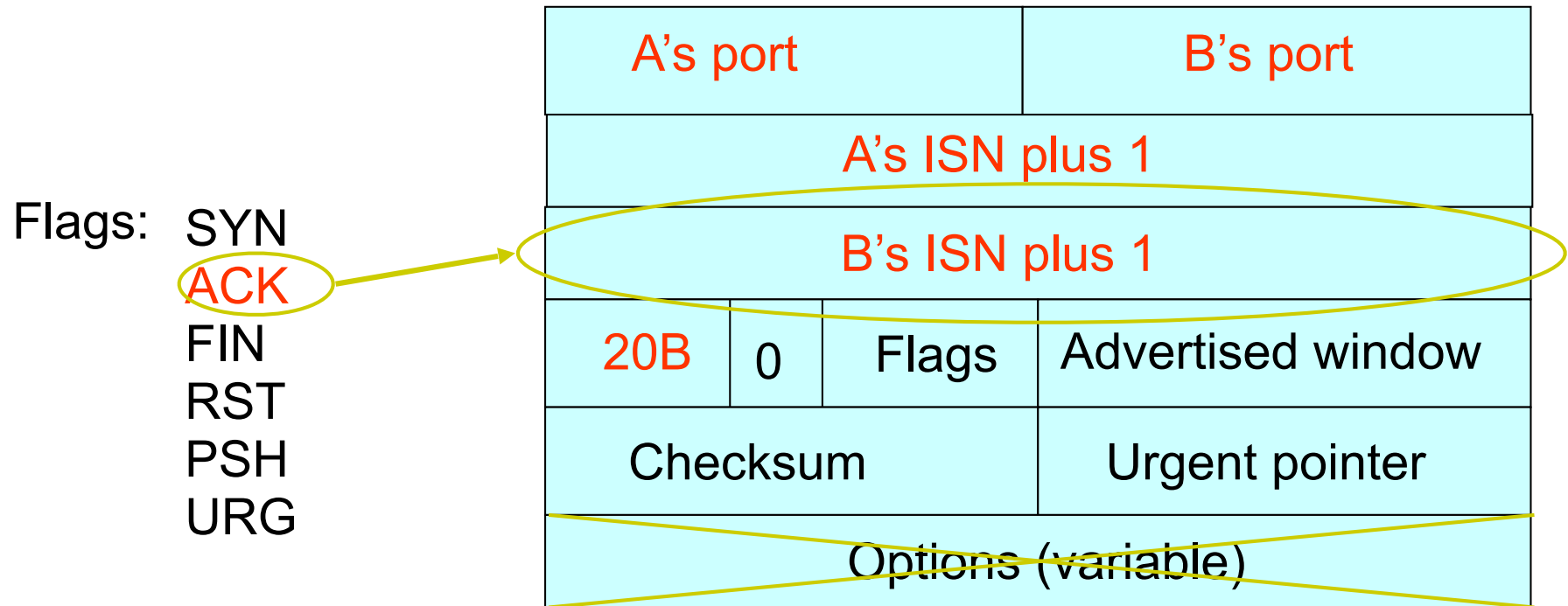
Step 2: B's SYN-ACK Packet



B tells A it accepts, and is ready to hear the next byte...

... upon receiving this packet, A can start sending data

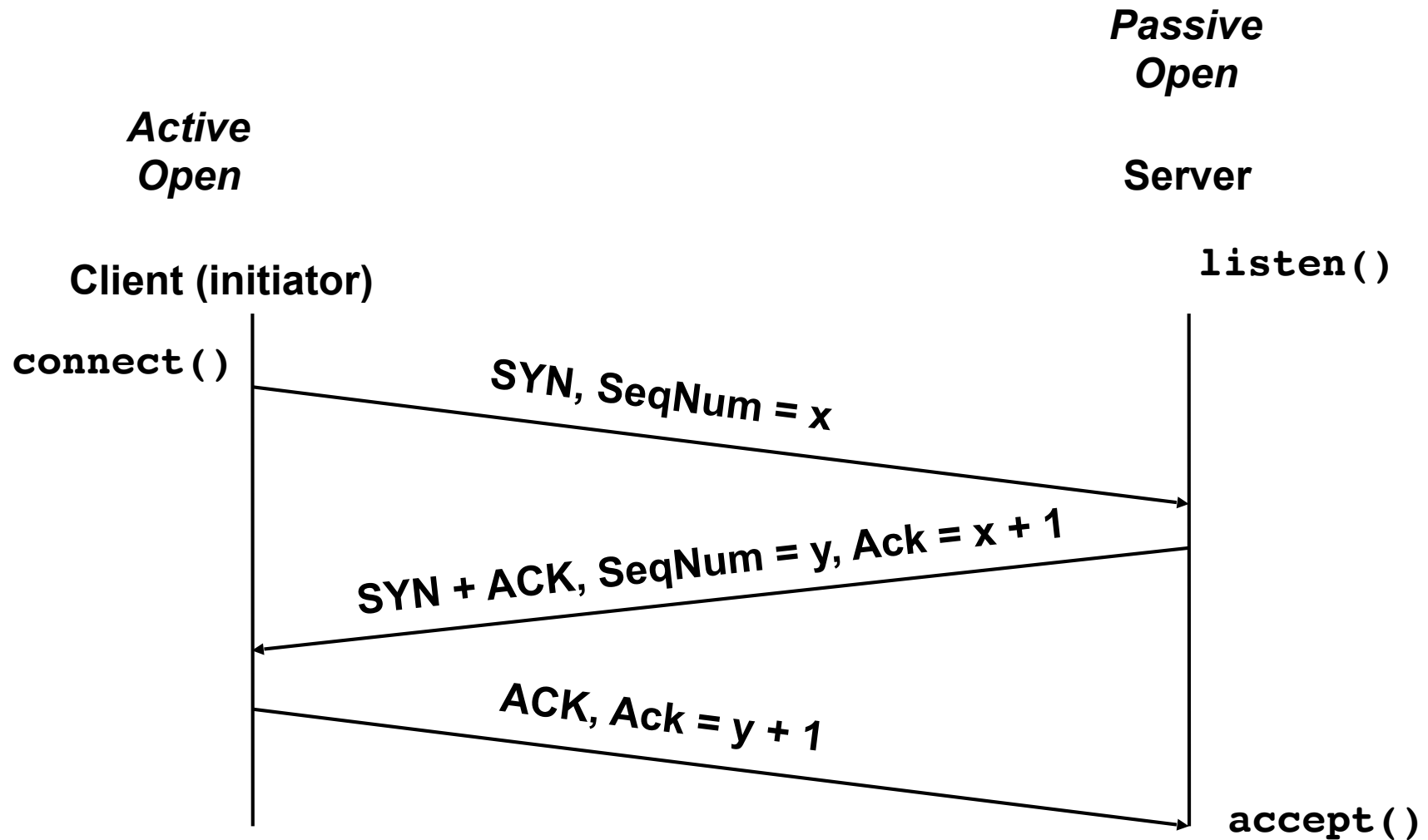
Step 3: A's ACK of the SYN-ACK



A tells B it's likewise okay to start sending

... upon receiving this packet, B can start sending data

Timing Diagram: 3-Way Handshaking



What if the SYN Packet Gets Lost?

Suppose the SYN packet gets lost

- Packet is lost inside the network, or:
- Server **discards** the packet (e.g., listen queue is full)

Eventually, no SYN-ACK arrives

- Sender sets a **timer** and **waits** for the SYN-ACK
- ... and retransmits the SYN if needed

How should the TCP sender set the timer?

- Sender has **no idea** how far away the receiver is
- Hard to guess a reasonable length of time to wait
- **SHOULD** (RFCs 1122 & 2988) use default of **3 seconds**
 - Other implementations instead use 6 seconds

SYN Loss and Web Downloads

User clicks on a hypertext link

- Browser creates a socket and does a “connect”
- The “connect” triggers the OS to transmit a SYN

If the SYN is lost...

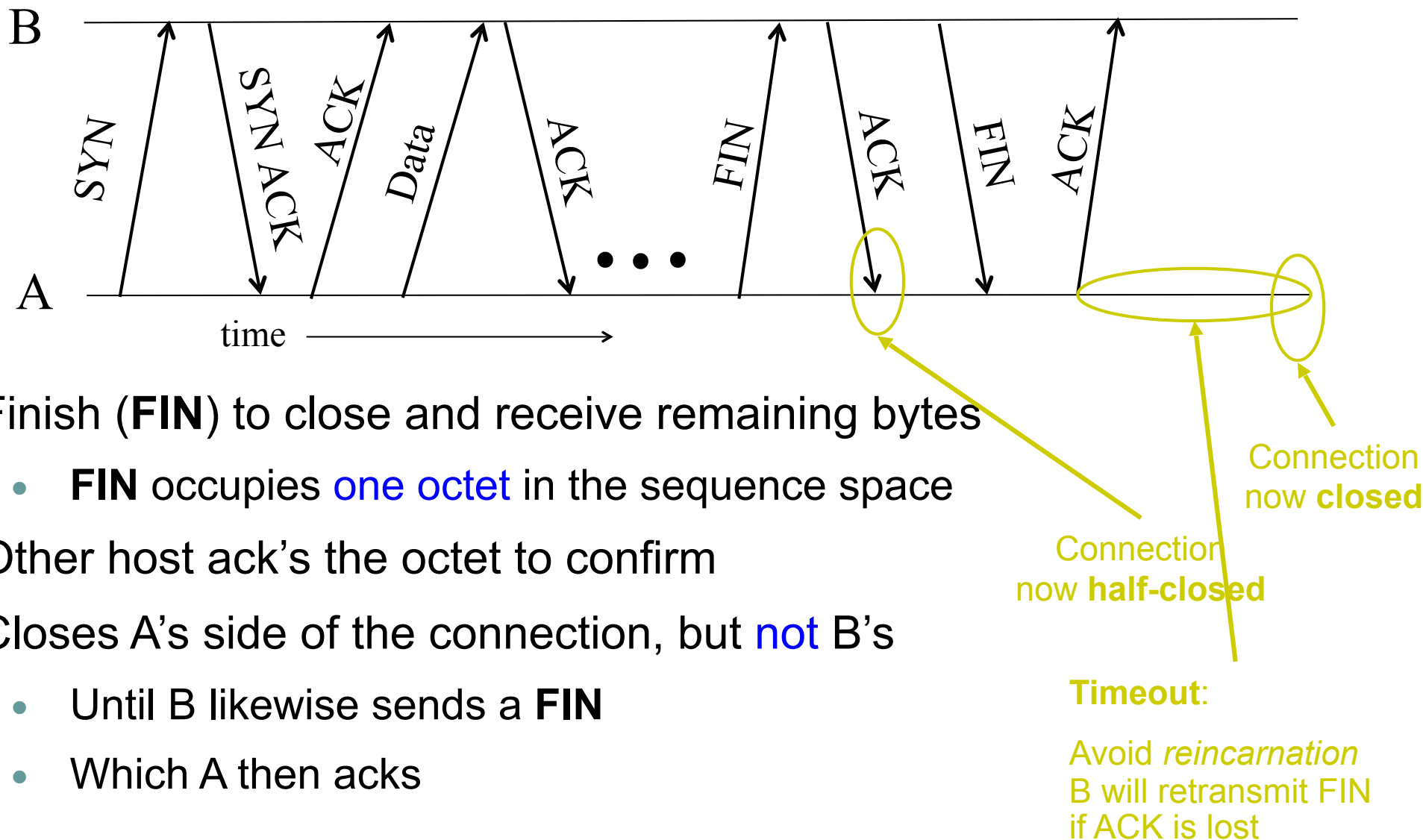
- 3-6 seconds of delay: can be **very long**
- User may become impatient
- ... and click the hyperlink again, or click “reload”

User triggers an “abort” of the “connect”

- Browser creates a **new** socket and another “connect”
- Essentially, forces a faster send of a new SYN packet!
- Sometimes very effective, and the page comes quickly

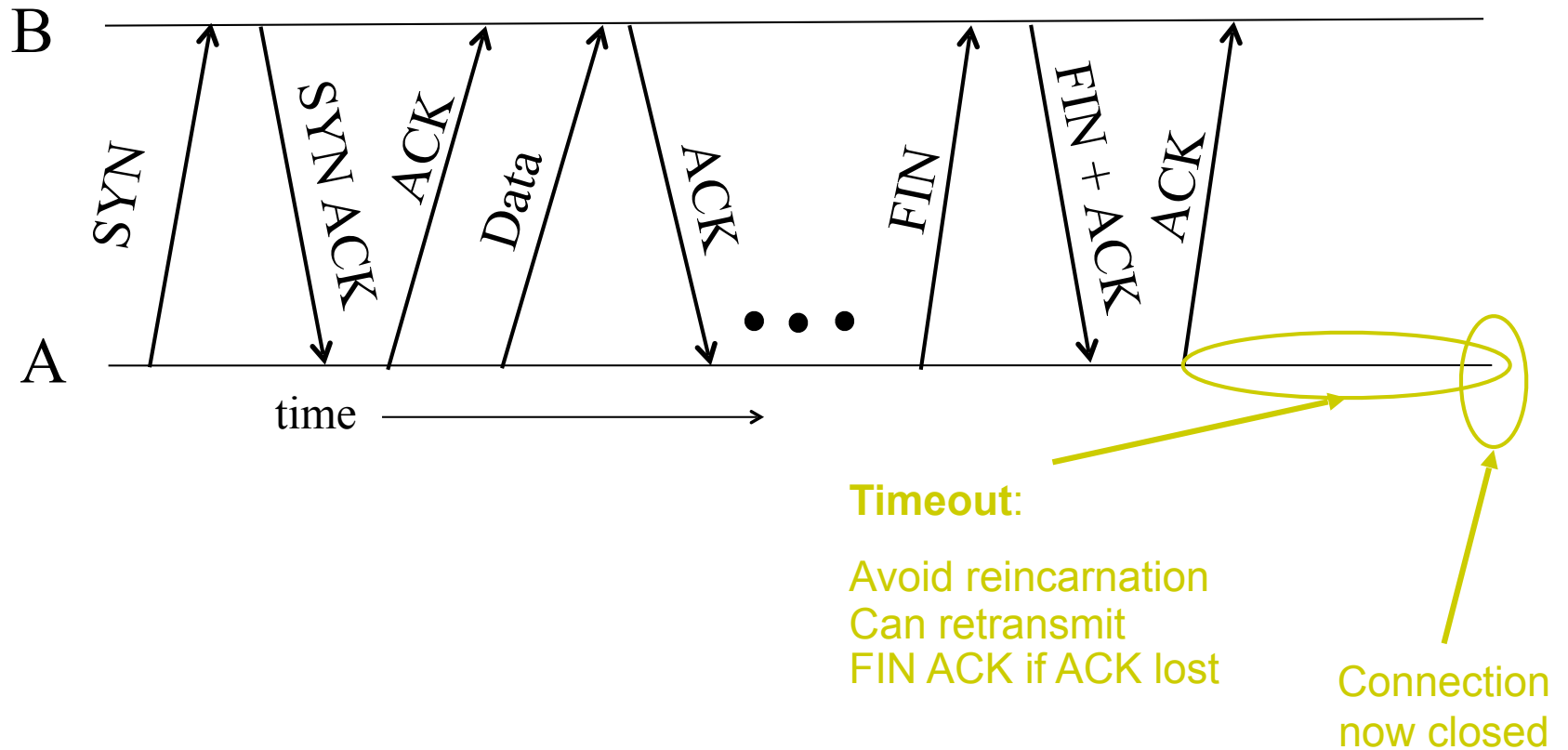
Tearing Down the Connection

Normal Termination, One Side At A Time

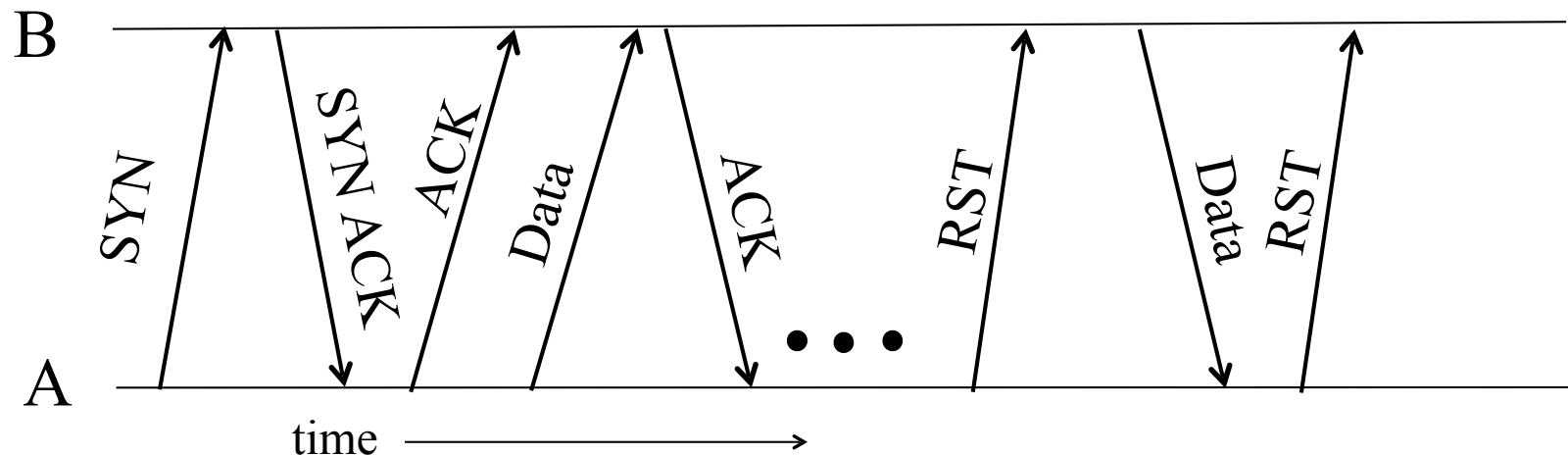


Normal Termination, Both Together

Same as before, but B sets **FIN** with their ack of A's **FIN**



Abrupt Termination



A sends a RESET (**RST**) to B

- E.g., because app. process on A **crashed**

That's it

- B does **not** ack the **RST**
- Thus, **RST** is **not** delivered **reliably**
- And: any data in flight is **lost**
- But: if B sends anything more, will elicit **another RST**

Reliability: TCP Retransmission

Timeouts and Retransmissions

Reliability requires retransmitting lost data

Involves setting timer and retransmitting on timeout

TCP resets timer whenever new data is ACKed

- Retx of packet containing “next byte” when timer goes off

Example

Arriving ACK expects 100

Sender sends packets 100, 200, 300, 400, 500

- Timer set for 100

Arriving ACK expects 300

- Timer set for 300

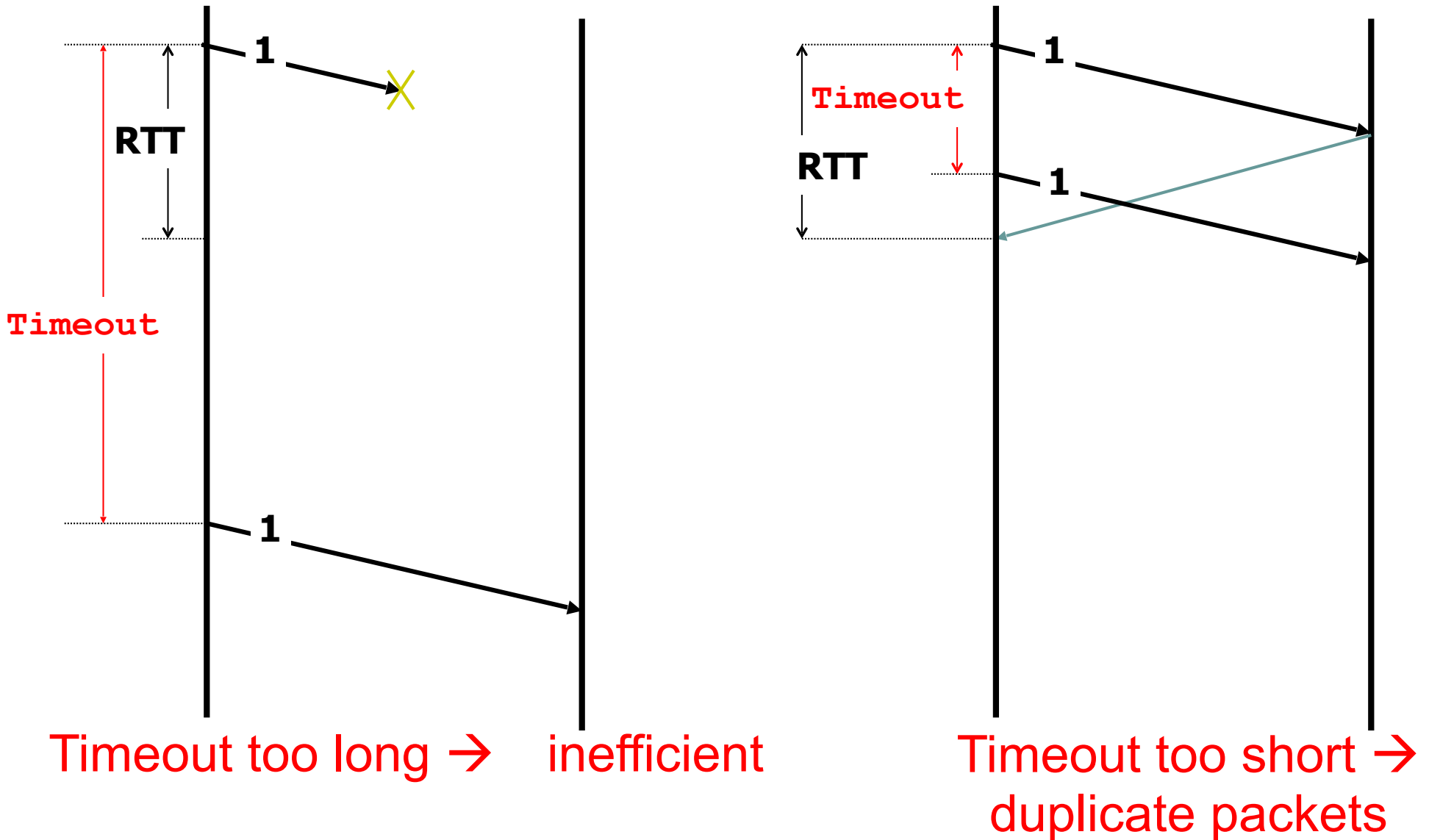
Timer goes off

- Packet 300 is resent

Arriving ACK expects 600

- Packet 600 sent
- Timer set for 600

Setting the Timeout Value



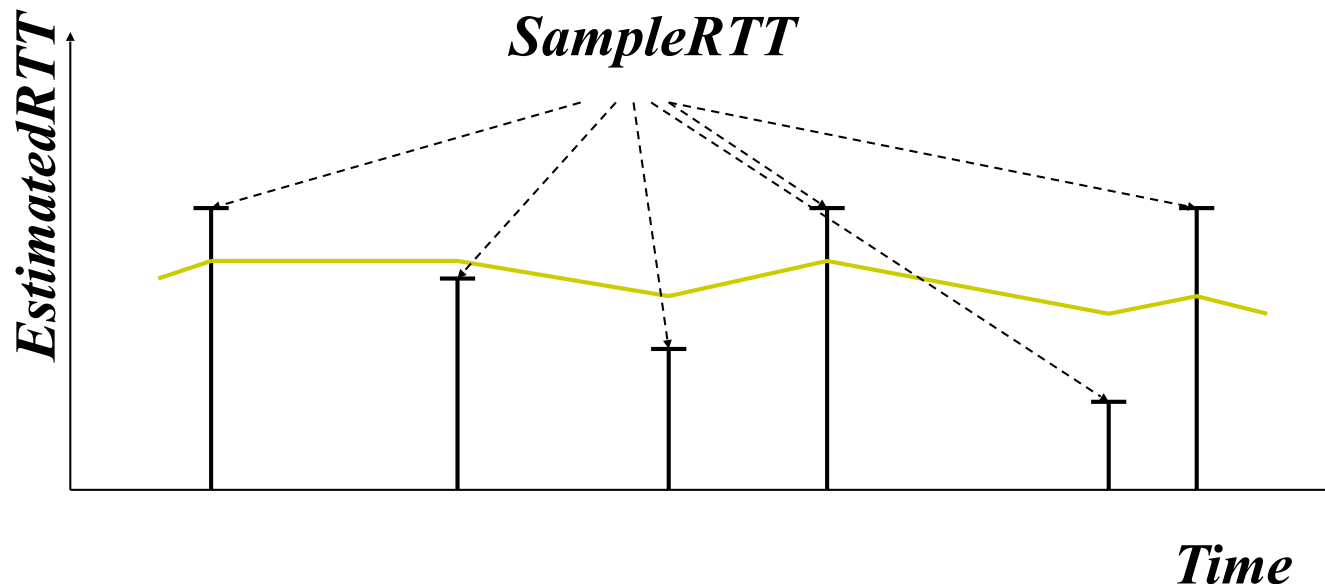
RTT Estimation

Use exponential averaging of RTT samples

$$\text{SampleRTT} = \text{AckRcvdTime} - \text{SendPacketTime}$$

$$\text{EstimatedRTT} = \alpha \times \text{EstimatedRTT} + (1 - \alpha) \times \text{SampleRTT}$$

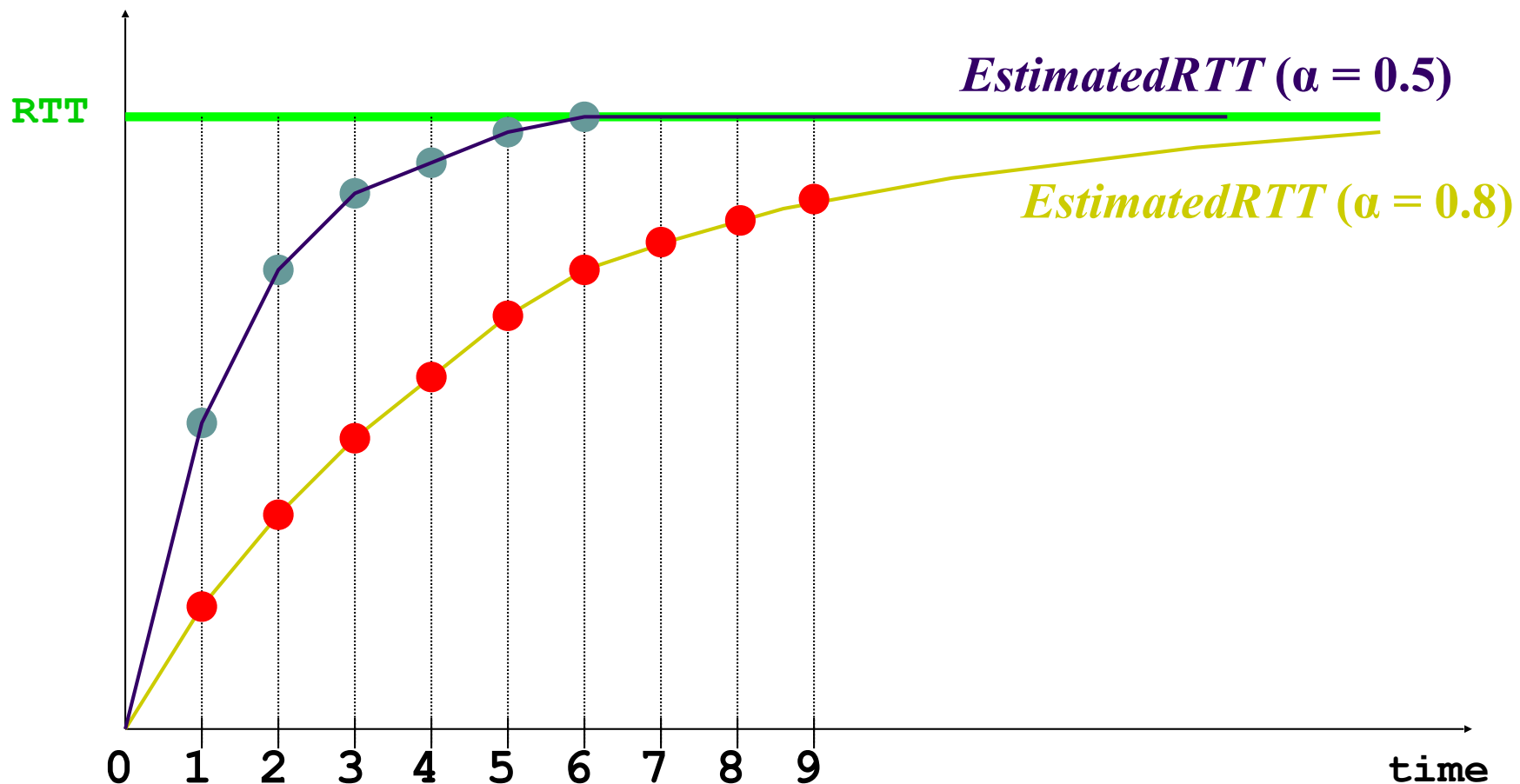
$$0 < \alpha \leq 1$$



Exponential Averaging Example

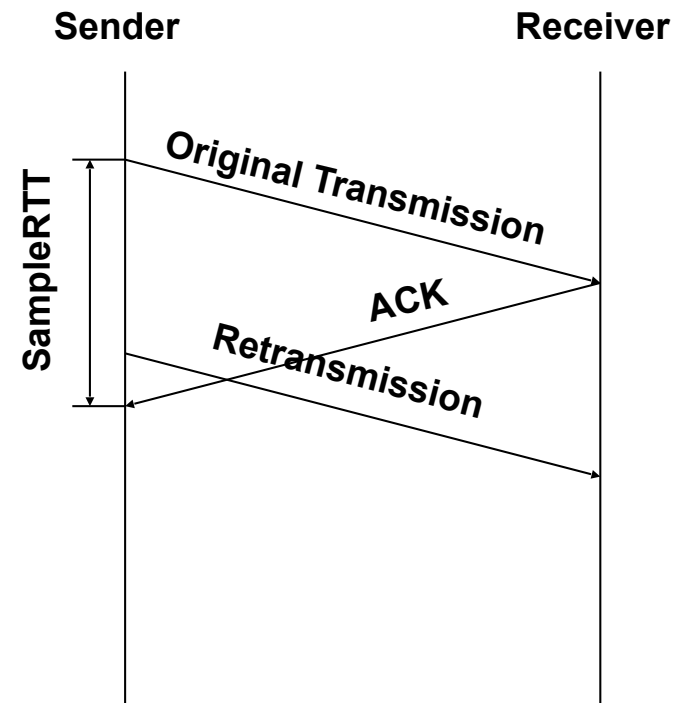
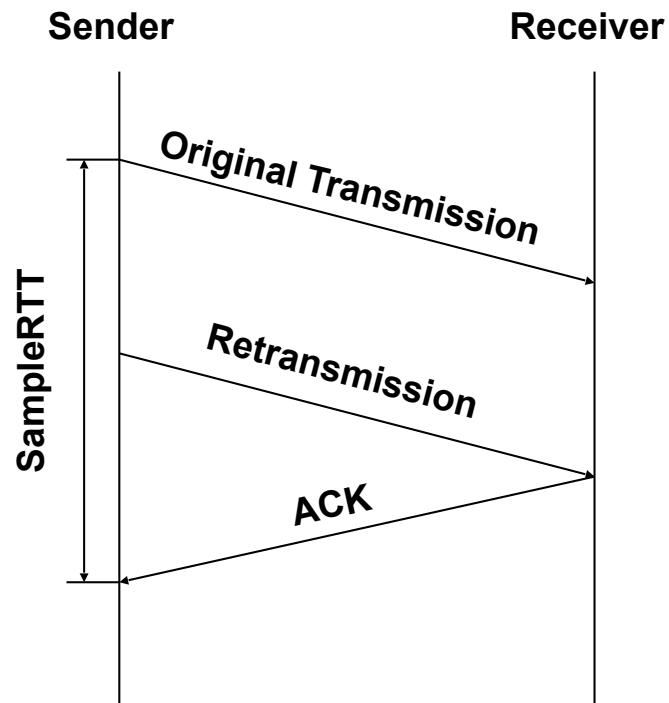
$$\text{EstimatedRTT} = \alpha * \text{EstimatedRTT} + (1 - \alpha) * \text{SampleRTT}$$

Assume RTT is constant $\rightarrow$ $\text{SampleRTT} = \text{RTT}$



Problem: Ambiguous Measurements

How do we differentiate between the real ACK, and ACK of the retransmitted packet?



Karn/Partridge Algorithm

Measure *SampleRTT* only for original transmissions

- Once a segment has been retransmitted, do not use it for any further measurements
- Computes *EstimatedRTT* using $\alpha = 0.875$

Timeout value (RTO) = $2 \times \text{EstimatedRTT}$

Use exponential backoff for repeated retransmissions

- Every time RTO timer expires, set $\text{RTO} \leftarrow 2 \cdot \text{RTO}$
 - (Up to maximum ≥ 60 sec)
- Every time new measurement comes in (= successful original transmission), collapse RTO back to $2 \times \text{EstimatedRTT}$

This is all very interesting, but.....

Implementations often use a coarse-grained timer

- 500 msec is typical

So what?

- Above algorithms are largely irrelevant
- **Incurring a timeout is expensive**

So we rely on duplicate ACKs

Loss with cumulative ACKs

Sender sends packets with 100B and seqnos.:

- 100, 200, 300, 400, 500, 600, 700, 800, 900, ...

Assume the fifth packet (seqno 500) is lost, but no others

Stream of ACKs will be:

- 200, 300, 400, 500, 500, 500, 500,...

Loss with cumulative ACKs

“Duplicate ACKs” are a sign of an *isolated* loss

- The lack of ACK progress means 500 hasn't been delivered
- Stream of ACKs means some packets are being delivered

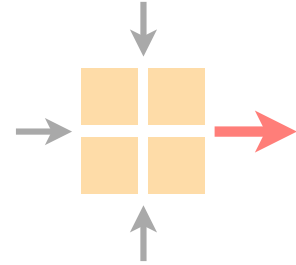
Therefore, could trigger resend upon receiving k duplicate ACKs

- TCP uses $k=3$

We will revisit this in congestion control

Communication Networks

Spring 2024



Laurent Vanbever

nsg.ee.ethz.ch

29 April 2024

D-ITET

Materials inspired from Scott Shenker & Jennifer Rexford